



The slide features a dark blue background with a large orange graphic in the top right corner containing a white plus sign and the number 26. The title 'Back To Basics' is in white, and 'Text Formatting' is in orange. The speaker's name 'BENJAMIN SAKS' is in orange. The Cppcon logo is in the bottom left, and the event details '2026 September 12 - 18' are in the bottom right.

Back To Basics
Text Formatting

BENJAMIN SAKS

 **Cppcon**
The C++ Conference

2026 | 
September 12 - 18

Saks & Associates

These notes are Copyright © 2026
in whole or part
by Ben Saks and Dan Saks
and distributed with their permission by:

Saks & Associates
393 Leander Drive
Springfield, OH 45504-4906 USA
+1-412-521-4117 (voice)
service@saksandassociates.com
saksandassociates.com

Recent Additions

- C provides text formatting facilities in the headers `<stdio.h>` and `<string.h>`.
- C++ provides its own versions of these headers, `<cstdio>` and `<cstring>`, which place their contents in the namespace `std`.
- C++ also provides its own text formatting tools, mostly in `<string>`, `<*stream>`, and `<iomanip>` headers.
- This talk is about C++'s new text formatting tools added in the headers `<format>` (C++20) and `<print>` (C++23).
- We'll start by reviewing those earlier libraries so we can see what advantages `<format>` and `<print>` provide.

7

Outline

- I/O Fundamentals
- Argument Indices and Dynamic Format Strings
- Formatting User-Defined Types

8

printf

- C++23's `<print>` header provides the function template `std::print`, as well as other related components.
- `std::print` is inspired by C's `printf` function (from `<stdio.h>`), which writes to standard output.
- The first argument to `printf` is called the ***format string***.
- The format string is usually (though not always) a string-literal.

9

printf

- In the simplest cases, the format string is the only argument to `printf`.
- For example, this program simply writes "hello, world" on a line by itself.

```
#include <stdio.h>

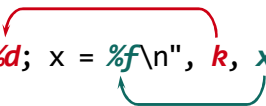
int main() {
    printf("hello, world\n");
    return 0;
}
```

10

Writing Values with printf

- `printf`'s first argument is called a format string because it can contain *format specifiers* such as `%d` and `%f`, used for writing the values of expressions.
- Each format specifier requires a corresponding additional argument:

```
printf("k = %d; x = %f\n", k, x);
```



- The `%d` specifier displays its corresponding argument as a decimal integer.
- The `%f` specifier displays its corresponding argument as a fixed-point number.

11

Fancier Formatting with printf

- Using the `%d` specifier by itself, `printf` displays exactly the number of decimal digits required for the value.
- However, you can alter how the number is displayed by changing the format specifier:

Integer Value	Format Specifier	Output Result
123	<code>%4d</code>	" 123"
123	<code>%-6d</code>	"123 "
49,374	<code>%x</code>	"c0de"
49,374	<code>%8X</code>	" C0DE"

12

Fancier Formatting with printf

- Using %f, printf displays exactly the number of digits required for the value, with 6 digits to the right of the decimal point.
- Here are some sample alternatives:

Numeric Value	Format Specifier	Output Result
12.347	%6.2f	" 12.35"
12.347	%-8.3f	"12.347 "
6.022 * pow(10, 23)	%e	"6.022000e+23"
6.022 * pow(10, 23)	%E	"6.022000E+23"

13

Writing Values with printf

- The %d format specifier is for int values, while the %f format specifier is for double values.
- If printf's format specifiers don't match their corresponding arguments, the behavior is undefined:

```
int k = 31;
double x = 19.2;
~~~
printf("k = %d; x = %f\n", x, k); // mismatching order
printf("k = %d; x = %f\n", k); // too few arguments
```

- Both of these printf calls have undefined behavior.
 - A modern C compiler might catch some mismatches, but it's not guaranteed.

14

I/O Streams

- C++ provides its own I/O library in `<iostream>` and related headers.
- One of the motivations for `<iostream>` was greater type safety.
- `<iostream>` relies heavily on operator overloading to achieve this type safety.

15

I/O Streams

- Here is equivalent code to write out stored values using both `printf` and the `<iostream>` library:

```
#include <stdio.h>
#include <iostream>

int k = 31;
double x = 19.2;
~~~
printf("k = %d; x = %f\n", k, x);
std::cout << "k = " << k << "; x = " << x << '\n';
```

- In this context, operator<< is called the ***stream insertion operation***, or simply an ***inserter***.

16

I/O Streams

- `<iostream>` uses `operator<<` to provide a uniform interface for writing string-literals, ints, doubles, and other values.
- To add support for a user-defined type `T`, just write an overloaded version of `operator<<` for `T`, as in:

```
std::ostream &operator<<(std::ostream &os, T const &t);
```

- Overload resolution automatically selects the correct `operator<<` function based on the types of its arguments.
- As such, you don't risk accidentally using the wrong format specifier for an object.

17

I/O Manipulators

- The Standard C++ header `<iomanip>` defines format controls called ***manipulators*** that control:
 - numeric precision,
 - left- and right-justification,
 - fill characters,
 - etc.

18

I/O Manipulators

- For example, `setw` is a manipulator that specifies field width, as in:

```
int age = 42;  
~~~  
std::cout << std::setw(4) << age;
```

- This displays `age` right-justified with leading spaces in a field that's at least 4 characters wide.
- `age` has type `int`, so this is equivalent to:

```
printf("%4d", age);
```

19

I/O Manipulators

- Whereas `setw` requires an argument, most manipulators do not.
- For instance, the following uses the hex manipulator to display numeric value `age` in hexadecimal format:

```
std::cout << std::hex << age;
```

20

I/O Manipulators

- Unfortunately, I/O manipulators are much more verbose than C's format specifiers:

```
int age = 42;
printf("%08X", age);           // prints "0000002A"
std::cout << std::uppercase << std::hex << std::setfill('0')
    << std::setw(8) << age;    // prints "0000002A"
```

- Fortunately, C++23's `<print>` header now allows us to write this instead:

```
std::print("{:08X}", age);    // prints "0000002A"
```

- Moreover, if `age` were not an integer, this call would produce a compile error.

21

std::format and std::print

- The headers `<format>` (C++20) and `<print>` (C++23) provide the terseness of C's format specifiers with the type safety and extensibility of C++ I/O streams.

Library	Terse	Type-Safe	Extensible
<code><stdio.h></code>	✓		
C++ streams		✓	✓
<code><format></code> <code><print></code>	✓	✓	✓✓

- In fact, `<format>` and `<print>` improve on C++ streams by making it easier to support their equivalent to I/O manipulators.

22

`std::format` and `std::print`

- `std::print` (from `<print>`) is strongly based on `std::format` (from `<format>`), introduced in C++20.
- `std::format` doesn't output text directly.
- Instead, it returns a `std::string` object containing the formatted text:

```
std::string s = std::format("k = {}\n", k);
```

23

`std::format` and `std::print`

- You can think of `std::print` as a shorthand for outputting the string returned by `std::format`:

```
std::print("k = {}\n", k);
```

// OR:

```
std::string s = std::format("k = {}\n", k);  
std::cout << s;
```

- I'll use `std::print` for many examples in this presentation.
- However, most examples would work equally well with `std::format`.

24

std::format_to

- `<format>` offers a few variations on `std::format`, including `std::format_to`.

- `std::format_to` writes the formatted text to an output iterator:

```
char buffer[20];
char *format_end = std::format_to(buffer, "k = {}", k);
*format_end++ = '\n';
*format_end++ = '\0';
std::cout << buffer;
```

- Like `std::copy` and other STL algorithms from `<algorithm>`, `std::format_to` returns an output iterator to the end of the formatted text.
- This code uses that behavior to append a newline character to the end.

25

Replacement Fields

- As with `printf`, the first argument to `std::print` is called the ***format string***:

```
int k = 3;
std::print("k = {}\n", k);
```

- The format string specifies how the values will be formatted into text.
- Note that `std::print`'s format strings are slightly different from `printf`'s format strings.

26

Replacement Fields

- This format string normally contains one or more *replacement fields* that specify where `std::print` will insert the values of the other arguments:

```
int k = 3;
std::print("k = {} \n", k); // prints "k = 3 \n"
```

- `{}` is the most basic form of replacement field; we'll examine others shortly.
- Like the `<iostream>` library, `std::print` is type-safe.
- It automatically recognizes that `k` is of type `int` and applies the default formatting for an `int`.

27

Replacement Fields

- If the format string contains multiple replacement fields, `std::print` normally inserts the other argument values in order:

```
int k = 3;
double x = 0.7;
std::string name = "Dan";

std::print("{} {} {}", k, x, name); // prints "3 0.7 Dan"
```

- Again, `std::print` recognizes the type of each argument and applies the appropriate formatting.

28

Replacement Fields

- If you need to print the literal characters { and }, the escape sequences are simply {{ and }} , respectively:

```
int k = 3;
double x = 0.7;
```

```
std::print("{{}}, {{}}", k, x);    // prints "{3, 0.7}"
```



29

Fancier Formatting

- A replacement field can also contain format specifiers that supersede the default format for the argument.
- A : character marks the beginning of a field's format specifiers.
- For example, this function formats an int to a field 6 characters wide (an int is right-aligned by default):

```
std::print("{:6}", 42);           // prints "    42"
```

30

Fancier Formatting

- Like `printf` and operator `<<`, `std::format` and `std::print` always write the full value, even if it doesn't fit in the specified width:

```
std::print("{:6}", 12345678); // prints "12345678"
```

31

Fancier Formatting

- For floating-point values, you can also specify precision, using `.` as a separator:

```
std::print("{:6.4}", 5.0 / 3); // prints " 1.667"
```

- Specifying precision for an object that isn't a floating-point number throws a `std::format_error`.
- If the format string is known at compile time, that exception becomes a compile error.

32

Fancier Formatting

- You can specify how a field should be aligned:
 - < means left-aligned
 - > means right-aligned
 - ^ means center-aligned (something printf doesn't support)

```
std::print("{:<6}", 42);    // prints "42   "
std::print("{:>6}", 42);    // prints "    42"
std::print("{:^6}", 42);    // prints "  42  "
std::print("{:^7}", 42);    // prints "   42   "
```

33

Fancier Formatting

- By default, any extra space in the field is filled with space characters.
- You can specify a different fill character immediately before the alignment specifier (if any):

```
std::print("{:*<6}", 42);    // prints "42****"
std::print("{:0>6}", 42);    // prints "000042"
std::print("{:-^6}", 42);    // prints "--42--"
std::print("{:-^7}", 42);    // prints "--42---"
```

34

Fancier Formatting

- After any fill, alignment, and width specifiers, you can place type-specific format specifiers.
- For example, you can format integral values in decimal, binary, octal, or hex:

```
std::print("{} ", 42);    // decimal, "42"
std::print("{:b} ", 42); // binary,  "101010"
std::print("{:o} ", 42); // octal,   "52"
std::print("{:x} ", 42); // hex,    "2a"
```

- Applying type-specific format specifiers to an incompatible type (e.g., x for a string) throws a `std::format_error` (possibly producing a compile error).

35

Range Formatting

- A replacement field can also refer to an entire range, such as an array or a vector:

```
std::array v1 {1, 2, 3, 4};
std::vector v2 {5, 6, 7, 8};
std::print("{} ", v1);    // prints "[1, 2, 3, 4]"
std::print("{} ", v2);    // prints "[5, 6, 7, 8]"
```

36

Range Formatting

- Range types handle width and alignment much like other types:

```
std::array v1 {1, 2, 3, 4};
std::vector v2 {5, 6, 7, 8};
std::print("{:>16}", v1);      // prints "      [1, 2, 3, 4]"
std::print("{:<16}", v2);      // prints "[5, 6, 7, 8]      "
```

37

Range Formatting

- The `n` format specifier removes the `[]` surrounding the elements:

```
std::array v3 {1, 2, 3, 4};
std::print("{:n}", v3);        // prints "1, 2, 3, 4"
```

- The `s` format specifier displays a range of characters as if it were a single string:

```
std::list v4 {'D', 'a', 'n'};
std::print("{:s}", v4);        // prints "Dan"
```

38

Range Formatting

- A format specifier after the first `:` applies to the entire range.
- You can use a second `:` to apply a format specifier to each of the individual elements within the range:

```
std::array v5 {8, 9, 10, 11};  
std::print!("{}", v5);           // prints "[8, 9, 10, 11]"  
std::print("{}:2", v5);         // prints "[ 8, 9, 10, 11]"  
std::print("{}:b", v5);         // prints "[1000, 1001, 1010, 1011]"
```

39

Formatting Other Types

- This is just a selection of the format specifiers and types that `std::format` and `std::print` accept.
- Notably, the `std::chrono` types support a wide range of format specifiers for date and time information.

40

Outline

- I/O Fundamentals
- Argument Indices and Dynamic Format Strings
- Formatting User-Defined Types

41

Indexing Replacement Fields

- Traditionally, text with embedded variables is difficult to translate.
- To borrow an example from Zverovich [2019], suppose that you're writing an application that displays the number of characters in a string.
- Furthermore, you need this application to support both the English and German languages....

42

Indexing Replacement Fields

- Given the string "Daniel", the English version of the program should display the following output:

String *"Daniel"* has 6 characters

- Given the same input, the German version should display:

6 Zeichen lang ist die Zeichenkette *"Daniel"*

- Note that the ordering of the word "Daniel" and the number 6 is opposite from the English version.

43

Indexing Replacement Fields

- Neither `printf` nor `<iostream>` provides a portable solution to this problem.
- Here's one possible `printf` format string for the English version:

"String \"%s\" has %u characters\n"

- And here's a possible `printf` format string for German:

%u Zeichen lang ist die Zeichenkette \"%s\" \n"

44

Indexing Replacement Fields

- You could *almost* support translation by writing your code to select the appropriate format string for the user's chosen language:

```
enum language {english, german};
constexpr language lang = language::english;
constexpr char const *translation_table[] = {
    "String \"%s\" has %u characters\n",
    "%u Zeichen lang ist die Zeichenkette \"%s\"\n"
};
printf(translation_table[lang], name, std::strlen(name));
```

- Unfortunately, since the two format strings order the variables differently, this code produces undefined behavior for the German version.

45

Indexing Replacement Fields

- With `std::format/std::print`, a replacement field can explicitly refer to a specific argument through an index:

```
enum language {english, german};
constexpr language lang = language::english;
constexpr char const *translation_table[] = {
    "String \"{0}\" has {1} characters\n",
    "{1} Zeichen lang ist die Zeichenkette \"{0}\"\n"
};
std::print(translation_table[lang], name, name.Length());
```

46

Indexing Replacement Fields

- You can also combine explicit indices with format specifiers:

```
std::print("{0:02} {1:e}", k, x); // prints "03 7.000000e-01"
```

- If you use any explicit indices in a format string, all indices must be explicit — you can't mix implicit and explicit indices.

47

Thinking Fourth-Dimensionally

- Both `std::format` and `std::print` validate the format string at compile time.
- Thus, mistakes often result in compile errors.
- For example, GCC 16.2 displays this error message when a replacement field refers to an argument that doesn't exist:

```
int k = 3;
std::print("k = {1}\n", k);
```

```
error: call to consteval function 'std::basic_format_string<char,
int&>("k = {1}")' is not a constant expression
```

```
std::print("k = {1}", k);
```

~~~~~

48

## Thinking Fourth-Dimensionally

- On the other hand, this means that the format string for `std::format` and `std::print` must be a compile-time constant expression.
- In the English-German translation code, you may have noticed that the chosen language was specified in a `constexpr` object...

49

## Thinking Fourth-Dimensionally

- If you remove this `constexpr` qualifier, this code produces a compile error because the format string isn't fixed at compile time:

```
enum language {english, german};
constexpr language lang = language::english;
constexpr char const *translation_table[] = {
    "String \"{0}\" has {1} characters\n",
    "{1} Zeichen lang ist die Zeichenkette \"{0}\"\\n"
};
std::print(translation_table[lang], name, name.length());

error: call to consteval function 'std::basic_format_string<char,
int&>(translation_table[lang])' is not a constant expression
    std::print(translation_table[lang], name, name.length());
                ~~~~~^
```

50

## Thinking Fourth-Dimensionally

- If you plan to build the English and German versions of the application separately, you can simply declare the language constexpr.
- But what if you want to build a single version of the application in which the user selects a language at run time?
- For that, you'd need somewhat different code....

51

## std::dynamic\_format

- std::dynamic\_format is a C++26 function that converts a run-time value into a format string that can be used with std::format or std::print:

```
std::string foo(std::string const &field_name, int value) {
 std::string fmt_str (field_name + " = {0}\n");
 return std::format(std::dynamic_format(fmt_str), value);
}
```

- Note that std::dynamic\_format is used in tandem with std::format or std::print, not as a replacement.

52

## std::dynamic\_format

- Unlike most format strings used with `std::format` or `std::print`, the format string that `std::dynamic_format` returns is *not* checked at compile time.
- If a formatting function finds that a dynamic format string doesn't match the other arguments, it throws `std::format_error`.

53

## std::dynamic\_format

- The object that `std::dynamic_format` returns is neither copyable nor movable.
- Trying to store the result and use it later normally produces a compile error:

```
std::string foo(std::string const &field_name, int value) {
 std::string fmt_str =
 std::dynamic_format(field_name + " = {0}\n");
 return std::format(fmt_str, value); // error
}
```

✓ Pass the result of `std::dynamic_format` directly to `std::format` or `std::print`.

54

## std::dynamic\_format

- Here is the English-German translation code using `std::dynamic_format`:

```
enum language {english, german};
language lang = language::english;
constexpr char const *translation_table[] = {
 "String \"{0}\" has {1} characters\n",
 "{1} Zeichen lang ist die Zeichenkette \"{0}\"\\n"
};
std::print(std::dynamic_format(translation_table[lang]), name,
 name.length());
```

- There's another solution that works in C++20/23, but it's a bit more complex...

55

## std::vformat

- `std::vformat` (also from `<format>`) is a non-template alternative to `std::format` that doesn't perform a compile-time check.
- As such, `std::vformat` lets you use a format string that is not a constant expression.
- If the specified format string isn't valid for the arguments given, `std::vformat` throws a `std::format_error`.

56

## std::vformat

- Since `std::vformat` isn't a function template like `std::format`, it must handle its variadic arguments somewhat differently.
- Specifically, you must wrap the variadic arguments in a call to the function `std::make_format_args`:

```
std::string bar(std::string const &f1, std::string const &f2) {
 int v1 = 3;
 double v2 = 0.7;
 std::string fmt_str (f1 + " = {0}, " + f2 + " = {0}\n");
 return std::vformat(fmt_str, std::make_format_args(v1, v2));
}
```

57

## std::vformat\_to

- `std::vformat` also has a variant that allows you to write to an output iterator, `std::vformat_to`.
- Like its counterpart, `std::vformat_to` returns an output iterator to the end of the formatted text.

58

## std::make\_format\_args

- The arguments to std::make\_format\_args must be lvalues.
- That's because the result of std::make\_format\_args doesn't actually contain full copies of its arguments.
- Think of a container of pointers to objects that live outside of the container.

59

## std::make\_format\_args

- If you try to pass an rvalue to std::make\_format\_args, you'll get a compile error:

```
T get_value(); // returns an rvalue
```

```
auto fmt_args = std::make_format_args(get_value()); // compile error
std::string s = std::vformat(fmt_str, fmt_args);
```

60

## std::make\_format\_args

- In general, you can work around this requirement by storing the value in a local object first:

```
T get_value(); // returns an rvalue
```

```
T temp = get_value();
auto fmt_args = std::make_format_args(temp); // OK
std::string s = std::vformat(fmt_str, fmt_args);
```

61

## Outline

- I/O Fundamentals
- Argument Indices and Dynamic Format Strings
- Formatting User-Defined Types

62

## Formatting User-Defined Types

- This class `rational` represents an exact fraction:

```
class rational {
public:
 rational(long n = 0, long d = 1);
 double as_double() const;
    ~~~
private:
    void reduce();
    long num, den;
};
```

- How do you print a `rational` object?

63

## Formatting User-Defined Types

- `<stdio.h>` doesn't provide a format specifier for `rational`.
- You can't extend `<stdio.h>` to provide format specifiers for user-defined types.
  - At least, you can't do it easily.
- You could add a member function to the class.
- But then printing a `rational` doesn't look at all like other output operations:

```
rational r (1, 3);
printf("The value is ");
r.print();
putchar('\n');
```

64

## Formatting User-Defined Types Using `<iostream>`

- You could print a `rational` using `<iostream>` by overloading `operator<<` for class `rational`:

```
class rational {
public:
    ~~~
 void put(std::ostream &os) const;
    ~~~
};

std::ostream &operator<<(std::ostream &os, rational const &r);
```

65

## Formatting User-Defined Types Using `<iostream>`

- Here are possible implementations for `put` and `operator<<`:

```
void rational::put(std::ostream &os) const {
    os << num;
    if (den != 1) {
        os << '/' << den;
    }
}

inline
std::ostream &operator<<(std::ostream &os, rational const &r) {
    r.put(os);
    return os;
}
```

66

## Formatting User-Defined Types Using `<iostream>`

- Now you can print a `rational` just like an object of a built-in type:

```
rational r (1, 3);
std::cout << "The value is " << r << '\n';
```

- Unfortunately, as written, `rational` doesn't really support I/O manipulators...

67

## Formatting User-Defined Types Using `<iostream>`

- Consider what happens if you try to print a `rational` using `setw(8)`:

```
rational r (1, 3);
std::cout << std::setw(8) << r << '\n';
```

- Here, the numerator will be printed with a width of 8, and then the denominator (if not 1) will be printed normally.

```
void rational::put(std::ostream &os) const {
    os << num; // setw(8) applies only to this line
    if (den != 1) {
        os << '/' << den;
    }
}
```

68

## std::formatter

- You can make *<format>* and *<print>* recognize a user-defined type by explicitly specializing the `std::formatter` template.
- `std::formatter` has two template type parameters:
  1. The user-defined type to be formatted (e.g., `rational`).
  2. The character type of the formatted text (e.g., `char`, `wchar_t`, etc.).
- For this talk, we'll use the default of `char` as the character type.
- Let's start with an explicit `std::formatter` specialization that just supports a default format...

69

## std::formatter

- An explicit specialization of `std::formatter` for `rational` should be defined shortly after the `rational` class definition:

```
#include <format>

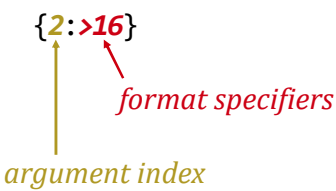
class rational {
    ~~~
};

// explicit specialization
template <>
struct std::formatter<rational, char> {
 // see following slides
};
```

70

## std::formatter

- The `std::formatter` specialization has two required member functions.
- The first is `parse`, which reads the format specifiers of a replacement field and figures out how to format the data:

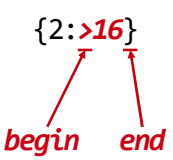


- `std::format` and `std::print` handle the argument index and `:` separator automatically.

71

## std::formatter

- `parse` receives a reference to an object of type `std::format_parse_context` from `std::print`/`std::format`.
- `std::format_parse_context` has several member functions, only two of which are important for our purposes:
  - `begin`: Returns an input iterator to the beginning of the format specifiers (shown in red), and
  - `end`: Returns an input iterator to the end of the format specifiers.



72

## std::formatter

- The second required `std::formatter` member function is `format`.
- `format` receives two arguments from `std::print/std::format`:
  1. The object of type `T` that provides the data to be written, and
  2. A `std::format_context` object that tells `format` where and how to write its output (e.g., it provides locale information).
- `std::format_context::out` returns an output iterator to the location where `format` should begin writing.
- `format` must return an output iterator to the next available output location.

73

## std::formatter

- For example, consider the following function call:  

```
rational r1 (-1, 3);
std::print("r1 = {}\n", r1);
```
- This sequence of boxes represents the output written by the call to `std::print`:

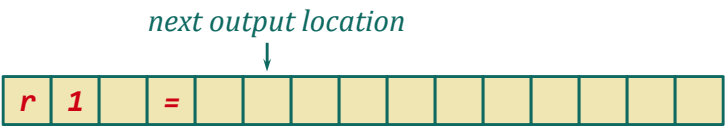


74

### std::formatter

```
rational r1 (-1, 3);
std::print("r1 = {}\\n", r1);
```

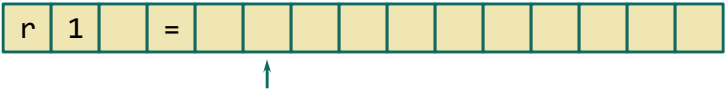
- First, std::print will write the literal characters from the format string itself, up to the first replacement field:



### std::formatter

```
rational r1 (-1, 3);
std::print("r1 = {}\\n", r1);
```

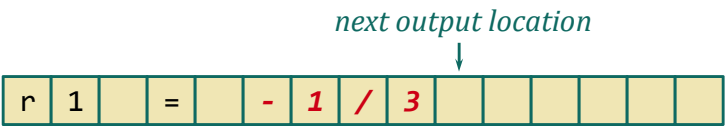
- When std::print reaches the replacement field, it will see that corresponding argument is of type rational (a user-defined type).
- Therefore, it will call std::formatter<rational, char>::format to write the value of r1.
- format receives a std::format\_context object containing this output iterator:



### std::formatter

```
rational r1 (-1, 3);
std::print("r1 = {} \n", r1);
```

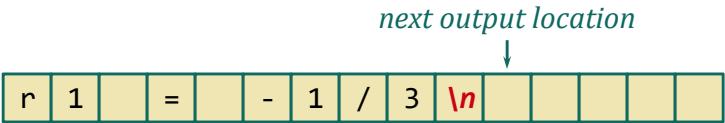
- Your code in `std::formatter<rational, char>::format` then writes out the characters for `-1/3`.
- `format` returns this output iterator, pointing just after the last character that it wrote (in this case, 3):



### std::formatter

```
rational r1 (-1, 3);
std::print("r1 = {} \n", r1);
```

- `std::print` then resumes writing from the iterator returned by `format`.
- In this case, it writes the `\n` character and then returns:



- Here's the general form of a simple `std::formatter` specialization, using type aliases to improve readability:

```
template <>
struct std::formatter<rational, char> {
 using p_context = std::format_parse_context;
 using f_context = std::format_context;

 constexpr p_context::iterator parse(p_context &ctx) {
 // see next slide
 }

 f_context::iterator
 format(rational const &r, f_context &ctx) const {
 // see later slides
 }
};
```

79

## Implementing `std::formatter`

- With just a default format, there's nothing for parse to interpret, so a simple placeholder will do.
- A minimal version of parse simply returns the iterator provided by the context:

```
constexpr p_context::iterator parse(p_context &ctx) {
 return ctx.begin();
}
```

80

## Implementing `std::formatter`

- Remember to make `parse` a `constexpr` function.
- `constexpr` will ensure that invalid replacement fields are caught at compile time (when the format string is a compile-time constant).

```
constexpr p_context::iterator parse(p_context &ctx) {
 return ctx.begin();
}
```

81

## Implementing `std::formatter`

- Since `std::format_to` writes to an output iterator, you can often write `format` in terms of a call to `std::format_to`.
- This implementation simply writes the numerator and denominator to the output iterator provided by the context:

```
f_context::iterator format(rational const &r, f_context &ctx) const {
 return std::format_to(ctx.out(), "{}/{}", r.num, r.den);
}
```

- With these basic implementations of `parse` and `format`, you have a working implementation of `std::formatter<rational, char>`.

82

## Implementing `std::formatter`

- `std::format` and `std::print` expect this `format` member function to be `const`.
- If you leave off this `const`, you'll probably see a compile error:

```
f_context::iterator format(rational const &r, f_context &ctx) const {
 return std::format_to(ctx.out(), "{}/{}", r.num, r.den);
}
```

83

## Implementing `std::formatter`

- Since `format` needs access to the `rational`'s `num` and `den` fields, we'll make the `std::formatter` specialization a friend:

```
class rational
{
    ~~~
private:
    friend struct std::formatter<rational, char>;
    long num = 0;
    long den = 1;
};
```

84

## Summary

- `std::format` and `std::print` offer the terseness of `printf` with the type safety and extensibility of C++ I/O streams.
- If the format string is a compile-time constant, most errors will be detected at compile time.
- If the format string is not a compile-time constant, you'll need to:
  - use C++26's `std::dynamic_format` when calling `std::format` or `std::print` (preferred), or
  - use a `v*` function like `std::vformat` or `std::vprint_unicode`, combined with `std::make_format_args`.

85

## Summary

- `std::format` and `std::print` handle user-defined types through specializations of the `std::formatter` struct template.
- Make `std::formatter<T>::parse` a `constexpr` function so it can detect errors at compile time.
- `std::formatter<T>::parse` must return an input iterator pointing to the closing `}` of the replacement field.
- `std::formatter<T>::format` must:
  - be a `const` member function, and
  - return an output iterator to the location where `std::format`/`std::print` will resume writing.

86

The End

Thanks for Listening

87

Just One More Thing...

88

## Second Pass at `std::formatter`

- With those basic implementations of `parse` and `format`, you have a working implementation of `std::formatter<rational, char>`.
- For many types, this may be enough.
- However, let's continue and add width support for `rational`...

89

## Second Pass at `std::formatter`

- If you have several `rational` values to write, you might want them to line up:

```
1/ 13
-3/ 175
29/ 401
```

- If `num` and `den` were public data members, the code might look like:

```
for (rational const &r : values) {
    std::print("{:>3}/{:>4}\n", r.num, r.den);
}
```

- Unfortunately, `num` and `den` are private...

90

## Second Pass at std::formatter

- Let's add width support for rational.
- Since you write the parse function, the format specifiers can be as simple or as complex as you like.
- Let's have the user will specify two widths separated by a / character, as in:

```
for (rational const &r : values) {
    std::print("{:3/4}\n", r);
}
```

- This way, the user can set widths for both the numerator and the denominator.

91

## Second Pass at std::formatter

- Now the std::formatter specialization needs data members to store the widths read by parse until format is called:

```
template <>
struct std::formatter<rational, char> {
    using p_context = std::format_parse_context;
    using f_context = std::format_context;
    short num_width = 1;
    short den_width = 1;
    ~~~
};
```

- Each time either std::format or std::print processes a replacement field for a rational, it creates a new std::formatter<rational, char> object.

92

## Second Pass at `std::formatter`

- First, the revised `parse` checks if the user has specified an explicit width:

```
constexpr p_context::iterator parse(p_context &ctx) {
 auto it = ctx.begin();
 if (std::isdigit(*it)) {
 // see upcoming slides
 }
 return it;
}
```

- If not, `parse` assumes the default format and simply returns the `begin` iterator, as in the previous case.

93

## Second Pass at `std::formatter`

- `parse` uses the helper function template `read_int` to read an integer from an input iterator.
- `read_int` returns a pair containing:
  - the integer it read, and
  - the input iterator, now pointing to the first non-digit from the start location.

```
template <std::integral T, std::input_iterator It>
constexpr std::pair<T, It> read_int(It b);
```

94

## Second Pass at std::formatter

- Here's the full implementation of read\_int:

```
#include <cctype>

template <std::integral T, std::input_iterator It>
constexpr std::pair<T, It> read_int(It i) {
 T value = 0;
 while (std::isdigit(*i)) {
 value *= 10;
 value += *i - '0';
 ++i;
 }
 return {value, i};
}
```

95

## Second Pass at std::formatter

- The helper function std::tie (from <tuple>) assigns the elements of a std::pair or std::tuple to existing objects:

```
std::pair<std::string, int> values ("Answer", 42);

std::string s;
int n;
std::tie(s, n) = values;
std::print("{} is {}\n", s, n); // prints "Answer is 42"
```

- The effect is similar to this line, except that s and n already exist:

```
auto [s, n] = values;
```

96

## Second Pass at std::formatter

- parse uses std::tie with read\_int to read in the numerator width while also advancing the input iterator:

```
constexpr p_context::iterator parse(p_context &ctx) {
 auto it = ctx.begin();
 if (std::isdigit(*it)) {
 std::tie(num_width, it) = read_int<short>(it);
 if (*it != '/') {
 throw std::format_error("expected '/' between widths");
 }
 ++it;
 std::tie(den_width, it) = read_int<short>(it);
 }
 return it;
}
```

97

## Second Pass at std::formatter

- The next character should be '/'; if it isn't, parse throws std::format\_error (possibly producing a compile error):

```
constexpr p_context::iterator parse(p_context &ctx) {
 auto it = ctx.begin();
 if (std::isdigit(*it)) {
 std::tie(num_width, it) = read_int<short>(it);
 if (*it != '/') {
 throw std::format_error("expected '/' between widths");
 }
 ++it;
 std::tie(den_width, it) = read_int<short>(it);
 }
 return it;
}
```

98

## Second Pass at std::formatter

- The next line simply advances beyond the '/' character:

```
constexpr p_context::iterator parse(p_context &ctx) {
 auto it = ctx.begin();
 if (std::isdigit(*it)) {
 std::tie(num_width, it) = read_int<short>(it);
 if (*it != '/') {
 throw std::format_error("expected '/' between widths");
 }
 ++it;
 std::tie(den_width, it) = read_int<short>(it);
 }
 return it;
}
```

99

## Second Pass at std::formatter

- Finally, this line reads the denominator width and advances the input iterator once again:

```
constexpr p_context::iterator parse(p_context &ctx) {
 auto it = ctx.begin();
 if (std::isdigit(*it)) {
 std::tie(num_width, it) = read_int<short>(it);
 if (*it != '/') {
 throw std::format_error("expected '/' between widths");
 }
 ++it;
 std::tie(den_width, it) = read_int<short>(it);
 }
 return it;
}
```

100

## Second Pass at `std::formatter`

- `parse` must return an input iterator that points to the closing `}` of the replacement field.
- If it doesn't, `std::format/std::print` won't know where to resume parsing the rest of the format string.
- The resulting error usually says something like “couldn't find closing `}`”.

101

## Second Pass at `std::formatter`

- The first implementation of `format` called `std::format_to` with a constant format string.
- This time, `format` must build the format string at run time using the `num_width` and `den_width` values.
- If `num_width` is 3 and `den_width` is 4, the resulting format string should look like this:

```
{:>3}/{:>4}
```

102

## Second Pass at std::formatter

- This version uses std::format to create that format string, using {{ and }} as escape sequences (highlighted below in amber):

```
f_context::iterator format(rational const &r, f_context &ctx) const {
 auto fmt_str =
 std::format("{{:}>{}}}/{{:}>{}}}", num_width, den_width);
 return std::vformat_to(ctx.out(), fmt_str,
 std::make_format_args(r.num, r.den));
}
```

- Again, the resulting format string looks like this:

```
{:>3}/{:>4}
```

103

## Second Pass at std::formatter

- Then it simply calls std::vformat\_to to do the real work, using the format string created on the previous line:

```
f_context::iterator format(rational const &r, f_context &ctx) const {
 auto fmt_str =
 std::format("{{:}>{}}}/{{:}>{}}}", num_width, den_width);
 return std::vformat_to(ctx.out(), fmt_str,
 std::make_format_args(r.num, r.den));
}
```

104

# The End

For Real This Time

105

## Bibliography

- Zverovich [2019]. Victor Zverovich, *Text Formatting*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html>

106